

CIA-3 PROJECT DEVELOPMENT — SUBMISSION REPORT

Christ University • 5th Semester • Advanced JavaScript Backend Frameworks

Field	Detail
Project Code & Title	P10 — Digital Banking Account & Transaction Management System
Domain	Banking & Financial Technology (Fintech)
Course Name	Advanced JavaScript Backend Frameworks (Node.js & Express JS)
Section / Batch	5 BTCS B
Semester	5th Semester
Team Number	Team 10

Team Details

S.No	Student Name	Register No.	Department	Section
1	PRARTHANA GURURAJ KATTI	2460495	B.Tech CSE	5 BTCS B
2	RIDHI M CHABBRIA	2460433	B.Tech CSE	5 BTCS B
3	ROSHEL MARIYA RIJO	2460438	B.Tech CSE	5 BTCS B
4	RICHARD RAJU	2460432	B.Tech CSE	5 BTCS B

GitHub Repository: <https://github.com/RichardRajuChiravath/L-T-5th-sem-Digital-Banking-Account-P10>

The repository is public and contains the complete Node.js + Express.js + MongoDB source code, a comprehensive README.md (project title, team details, module list, setup steps, API endpoints), a .gitignore, a .env.example, and the exported Postman collection. node_modules and .env are excluded from version control.

Table of Contents

Click any section below to navigate directly to that topic.

Section	Topic Description	Page
1. Project Overview	Problem Statement, Objectives, Scope & Boundaries, Technology Stack, Module Summary	3
2. System Architecture	Request Pipeline, MVC Project Structure, Response Envelope Design	5
3. Database Design	Collections, Reference vs. Embed Decisions, Indexes, ER Notes, State Machine	5
4. Functional Modules & Evidence	Detailed Breakdown of all 13 Functional Modules with Postman & UI Evidence	7
5. Complete API Endpoint Reference	Granular Method, Route, Auth Level & Description Reference Table	21
6. Testing, Validation & Error Handling	Negative & Edge Case Assertions, Schema Validation, Central Error Handler	22
7. Setup & Run Instructions	Step-by-step Installation, Environment Configuration, Pre-seeded Accounts	23
8. Known Limitations	Documented Boundaries & Assumptions	23
9. Learnings & Challenges	Technical Challenges, Concurrency, Double-Entry Accounting, RBAC Insights	24
10. Conclusion	Final Summary of Project Deliverables & Academic Compliance	24

1. Project Overview

The Digital Banking Account & Transaction Management System (P10) is a corporate-grade, resilient backend engineered for core-banking operations. Customers can open savings and current accounts, submit KYC credentials, execute atomic inter-account fund transfers, maintain trusted beneficiaries, and review point-in-time immutable ledger statements. Bank Staff review application queues, enforce Anti-Money Laundering (AML) controls on high-value transfers, and manage security freezes. Bank Admins oversee liquidity metrics and trigger periodic interest accrual batches.

1.1 Problem Statement

Conventional banking applications often suffer from database race conditions, lack of point-in-time transaction auditability, and poor role segregation between customer features and operational compliance. This project provides a dependable double-entry style ledger backend with atomic balance enforcement, strict account lifecycle state transitions (Pending -> Active -> Frozen), automated high-value suspicious transaction flagging, and granular Role-Based Access Control (RBAC).

1.2 Objectives

- Build an atomic fund transfer engine that eliminates double-spending and ledger inconsistencies.
- Implement an immutable transaction ledger recording running balanceAfter for every credit and debit.
- Establish a 3-tier RBAC architecture (CUSTOMER, STAFF, ADMIN) with JWT bearer token verification.
- Automatically flag high-value transactions (>= 50,000 INR) for compliance and AML review.
- Provide an interactive, responsive corporate web portal matching luxury banking aesthetics.

1.3 Technology Stack Justification

Component	Technology	Justification
Runtime	Node.js (v18+ LTS)	Non-blocking, event-driven I/O ideal for concurrent financial transaction throughput.
Web Framework	Express.js (v4.19)	Lightweight, unopinionated routing with composable middleware for JWT auth, validation, and logging.
Database	MongoDB Atlas + Mongoose (v8.3)	Flexible JSON document model for structured KYC and append-only ledgers; Mongoose schemas enforce data integrity.
Authentication	JWT + bcryptjs (v2.4)	Stateless Bearer token authorization; bcrypt salt rounds ensure safe credential storage.
Validation	Custom Validator Middleware	Centralized request body sanitizer returning standard 400 validation envelopes without crashing.
Frontend Portal	HTML5 + Vanilla CSS + Bootstrap 5	High-performance, elegant cream-gold luxury aesthetic with EB Garamond typography for live viva demos.

1.4 Module Summary

#	Module Name	Implementation Status
1	Customer Onboarding & KYC Capture	Implemented & Verified
2	Account Approval Workflow (Staff)	Implemented & Verified
3	Account Management (Savings & Current)	Implemented & Verified
4	Beneficiary Management (Trusted Payees)	Implemented & Verified
5	Fund Transfer Engine (Atomic Balances)	Implemented & Verified
6	Transaction Ledger (Running Balance)	Implemented & Verified
7	Account Statement Generation (Date-range filters)	Implemented & Verified

8	Minimum Balance & Limits Enforcement	Implemented & Verified
9	Suspicious Transaction Flagging (AML > 50,000 INR)	Implemented & Verified
10	Account Freeze / Unfreeze Module	Implemented & Verified
11	Interest Calculation Job Logic (Savings simple interest)	Implemented & Verified
12	Staff Monitoring Dashboard	Implemented & Verified
13	Role-Based Access Control (Customer, Staff, Admin)	Implemented & Verified

2. System Architecture

The system follows the Model-View-Controller (MVC) architectural pattern. Request handling is decoupled into composable layers: routing, authentication and RBAC guards, input validation, business logic controllers, Mongoose data schemas, and centralized error handling.

2.1 Request Pipeline

HTTP Request (Client / Hoppscotch / Web UI)

- CORS + express.json() Body Parsing
- Morgan HTTP Logger
- Route Match (routes/authRoutes.js, accountRoutes.js, etc.)
- protect (middleware/auth.js) -> verifies JWT, loads live user -> req.user
- authorize('STAFF', 'ADMIN') -> route-level RBAC check
- validateBody(rules) -> validates types, limits, required fields
- Controller (controllers/*.js) -> business rules & state transition
- Model (models/*.js) -> Mongoose schema + indexes -> MongoDB Atlas
- sendSuccess / sendError -> standardized JSON response envelope
- errorHandler (middleware/errorHandler.js) -> catches CastError, duplicate keys

2.2 Response Envelope Design

Every endpoint strictly produces a uniform JSON response structure to guarantee frontend reliability:

```
// Success Envelope (HTTP 200 / 201):
{  "success": true,   "message": "Status updated successfully",  "data": { ... } }

// Error Envelope (HTTP 400 / 401 / 403 / 404 / 409):
{  "success": false,  "message": "Requested action violates a business rule",  "errorCode":
"MINIMUM_BALANCE_VIOLATION" }
```

3. Database Design

Data modeling in MongoDB balances atomicity, query efficiency, and storage limits. Referencing and embedding decisions were made deliberately around the access patterns of core banking:

Entity / Field	Choice	Justification
kycDetails (in users)	Embedded	1-to-1 relationship; tightly coupled to customer identity and always read together upon login.
accounts	Referenced	1-to-many relationship; a customer maintains multiple independent accounts (Savings, Current) with separate lifecycles.
beneficiaries	Referenced	Kept in a standalone collection to prevent account document bloat; compound uniqueness on (userId, accountNo).
transactions (ledger)	Referenced (Immutable)	Append-only records grow into thousands of entries. Embedding would exceed MongoDB's 16MB document limit.
approvals	Referenced	Compliance audit trail recording decision history (Approve, Reject, Freeze, Unfreeze) with staff references.

3.1 Suggested Indexes

Collection	Index Definition	Optimization Purpose
users	{ email: 1 } (unique)	Guarantees unique logins and sub-millisecond user lookups.
accounts	{ userId: 1 }	Accelerates customer dashboard queries fetching owned accounts.

accounts	{ accountNumber: 1 } (unique)	Enforces unique account numbers and speeds up fund transfer routing.
beneficiaries	{ userId: 1, beneficiaryAccountNumber: 1 }	Prevents adding duplicate transfer payees.
transactions	{ accountId: 1, createdAt: -1 }	High-speed query for account statements and ledger history.
transactions	{ flagged: 1 }	Instant filtering of AML suspicious transactions for staff review.

4. Functional Modules & Execution Evidence

Each of the 13 required modules is documented below with its core endpoints, business rules, and real execution screenshots from Hoppscotch & the web portal.

4.1 Module 1 — Customer Onboarding & KYC Capture

Public registration capturing personal details and structured KYC credentials (panNumber, aadhaarNumber, address, annualIncome). Passwords are salted and hashed using bcryptjs. Returns a signed JWT token upon successful onboarding.

Method	Route	Access	Description
POST	/api/auth/register	Public	Customer registration with KYC
POST	/api/auth/login	Public	User authentication returning JWT
GET	/api/auth/me	Authenticated	Current logged-in profile

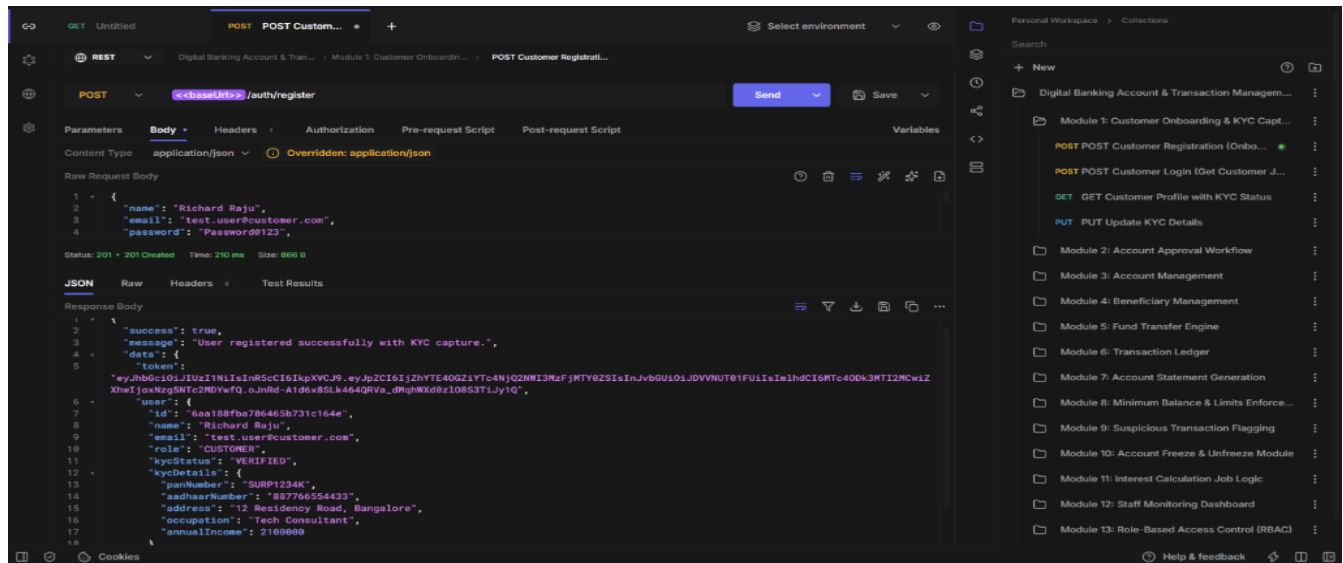


Figure Evidence: Customer Registration & KYC Capture — `POST /api/auth/register` -> 201 Created with JWT token and KYC profile

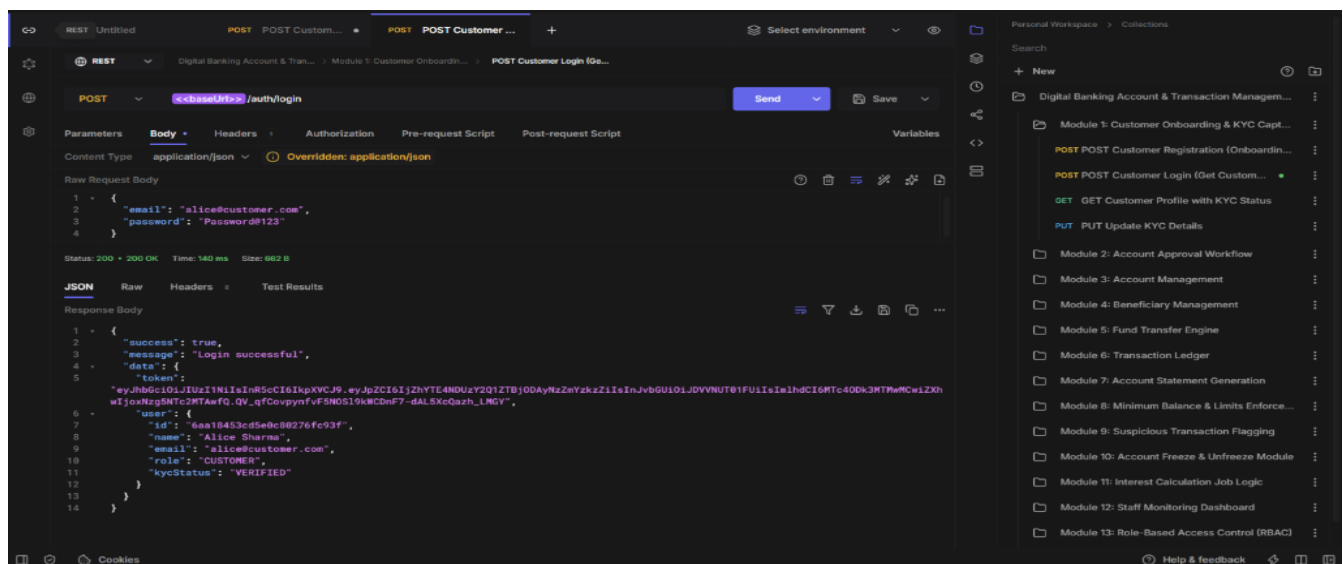


Figure Evidence: Customer Login & JWT Token Generation — `POST /api/auth/login` -> 200 OK with Bearer token and user payload

4.2 Module 2 — Account Approval Workflow

Core banking compliance workflow. Account applications start in PENDING_APPROVAL status. Authorized Bank Staff inspect the queue and execute approve or reject decisions with mandatory audit remarks.

Method	Route	Access	Description
GET	/api/staff/pending-accounts	Staff/Admin	View pending applications
PUT	/api/accounts/:id/approve	Staff/Admin	Approve account application
GET	/api/staff/audit-trail/:id	Staff/Admin	View approval decision trail

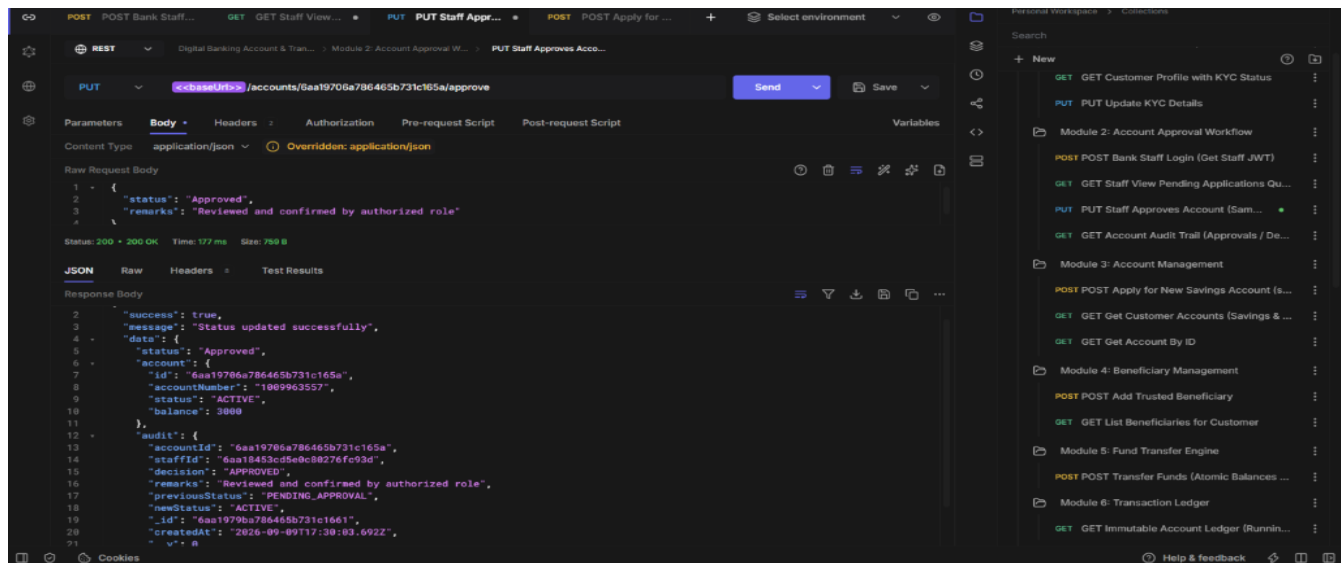


Figure Evidence: Staff Approves Account Application — PUT /api/accounts/:id/approve -> 200 OK (Status: Approved, initial deposit activated)

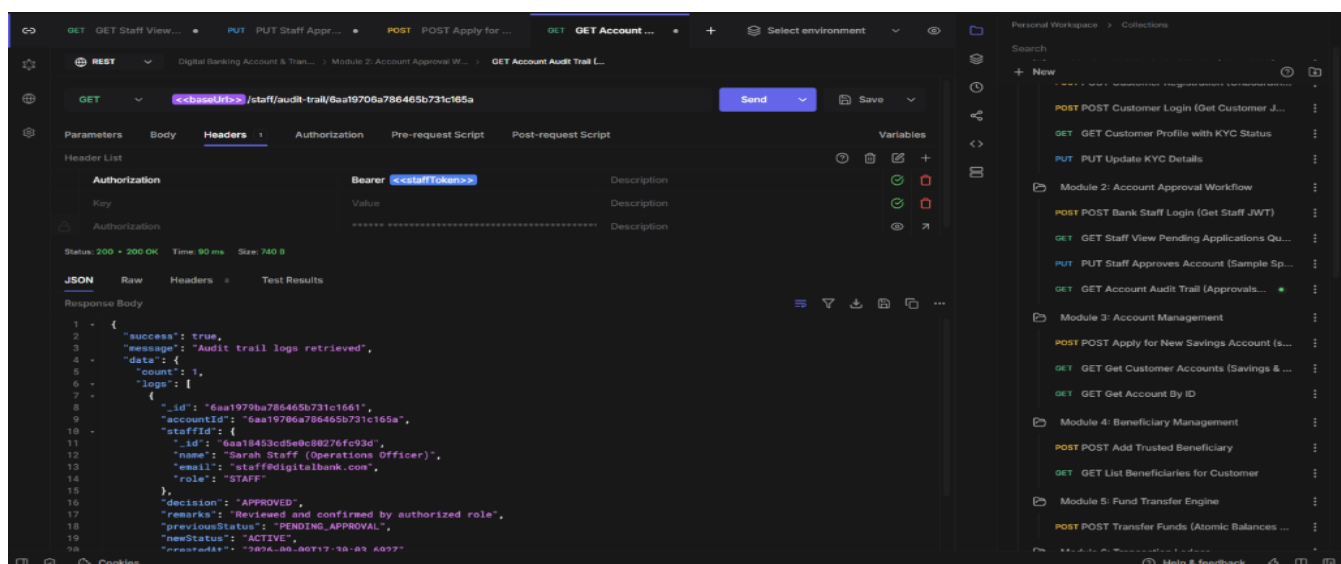


Figure Evidence: Immutable Approval Audit Trail Record — GET /api/staff/audit-trail/:id -> 200 OK with staff decision history

4.3 Module 3 — Account Management

Management of SAVINGS and CURRENT accounts. Enforces initial deposit rules, tracks live balances, daily limits, and account statuses (PENDING_APPROVAL, ACTIVE, FROZEN, REJECTED).

Method	Route	Access	Description
POST	/api/accounts	Customer	Apply for Savings/Current account
GET	/api/accounts	Customer/Staff	List accounts with balances
GET	/api/accounts/:id	Customer/Staff	Retrieve account by ID

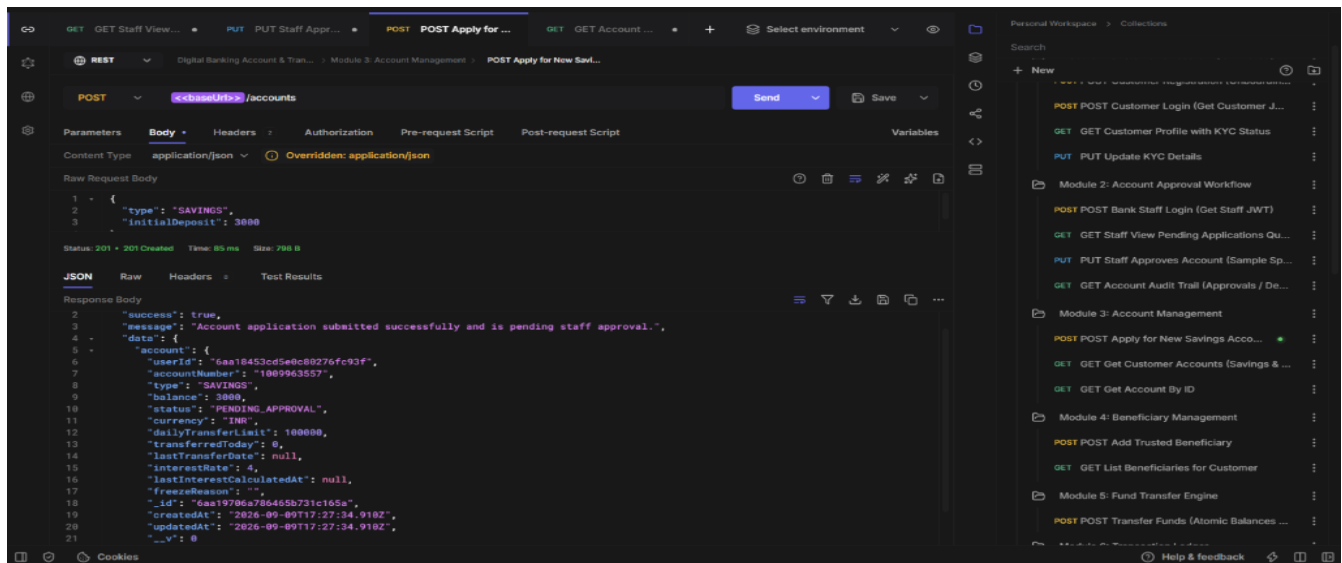


Figure Evidence: Apply for New Savings Account — `POST /api/accounts` -> 201 Created (Status: `PENDING_APPROVAL`, balance: 3,000)

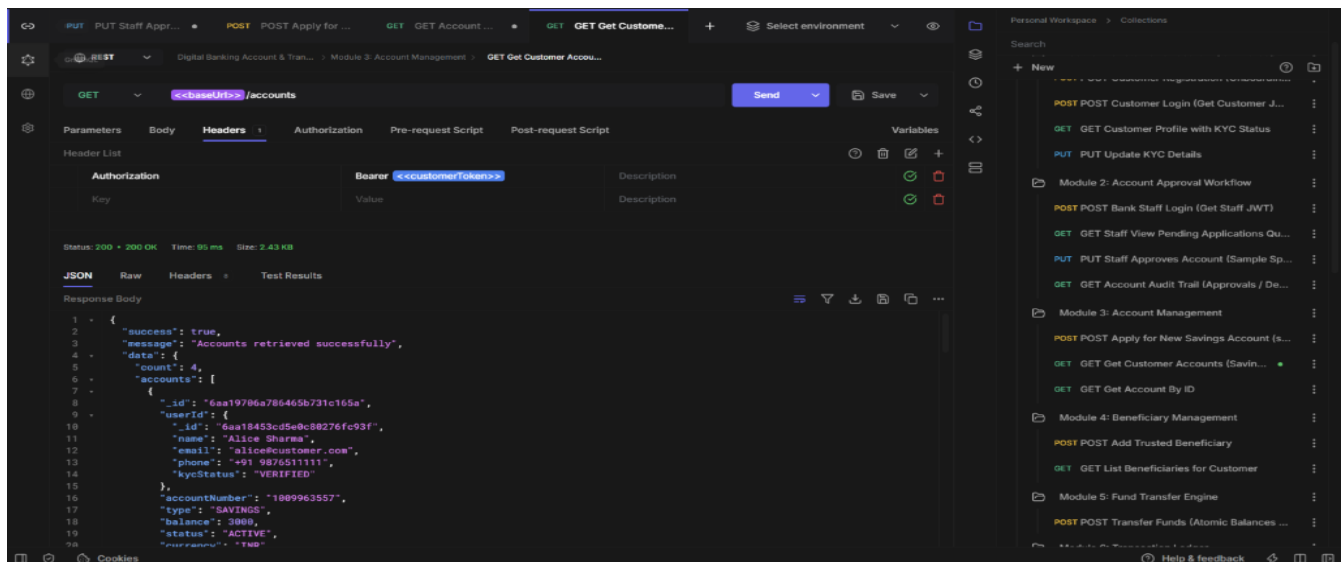


Figure Evidence: Get Customer Accounts with Live Balances — `GET /api/accounts` -> 200 OK showing Savings & Current accounts

4.4 Module 4 — Beneficiary Management

Customers register trusted counterparties for streamlined fund transfers. Validates source account ownership, rejects self-transfers, and enforces compound unique indexing to eliminate duplicate payees.

Method	Route	Access	Description
POST	/api/beneficiaries	Customer	Add trusted beneficiary
GET	/api/beneficiaries	Customer	List saved beneficiaries
DELETE	/api/beneficiaries/:id	Customer	Remove a beneficiary

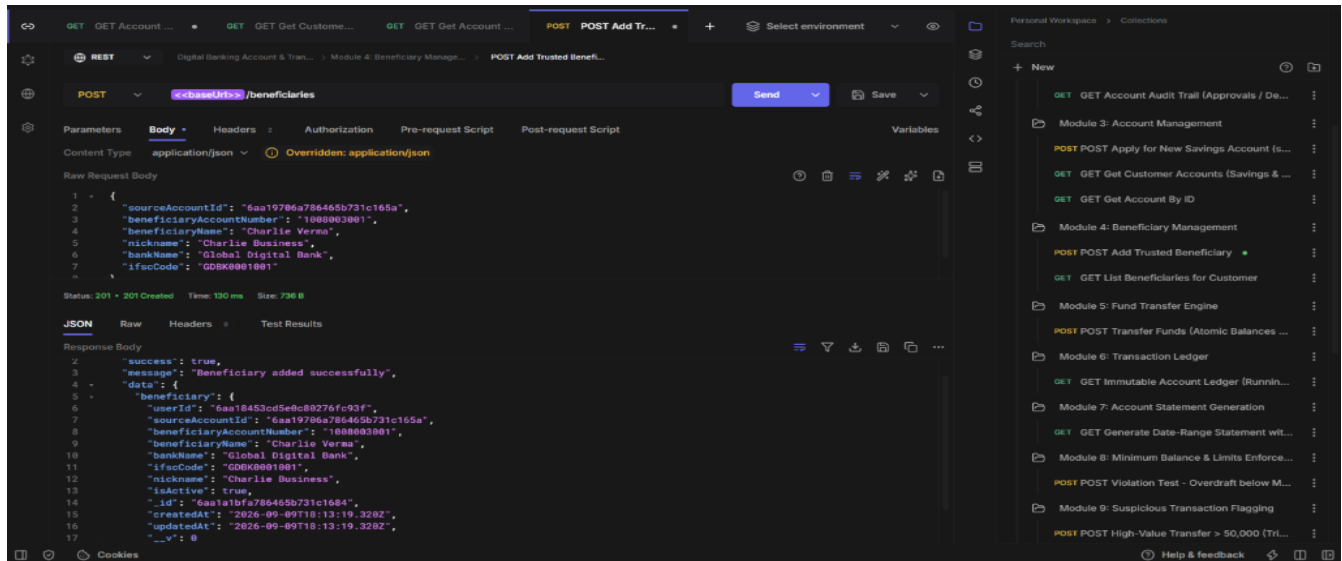


Figure Evidence: Add Trusted Counterparty Beneficiary — POST /api/beneficiaries -> 201 Created with linked account details

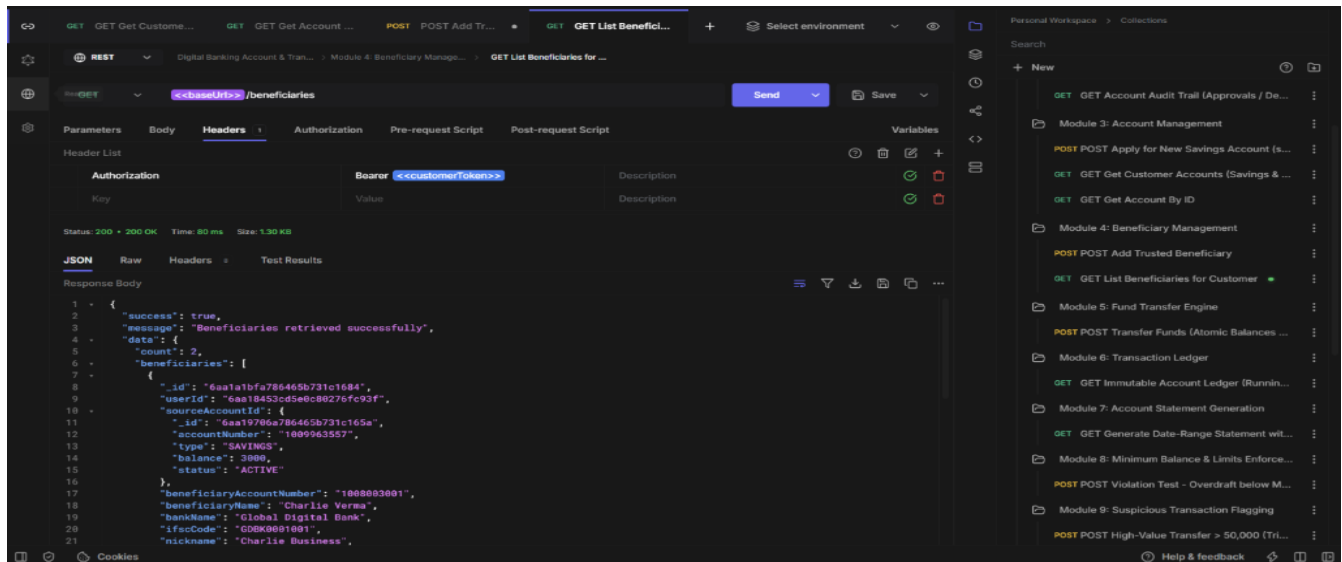


Figure Evidence: List Verified Beneficiaries — GET /api/beneficiaries -> 200 OK with active beneficiary array

4.5 Module 5 — Fund Transfer Engine

High-integrity transfer engine executing atomic balance debit from sender and credit to receiver. Pre-validates that neither account is frozen, checks minimum balance thresholds, and logs dual ledger entries.

Method	Route	Access	Description
POST	/api/transactions/transfer	Customer	Atomic inter-account transfer
POST	/api/transactions/deposit	Authenticated	Cash counter deposit simulation

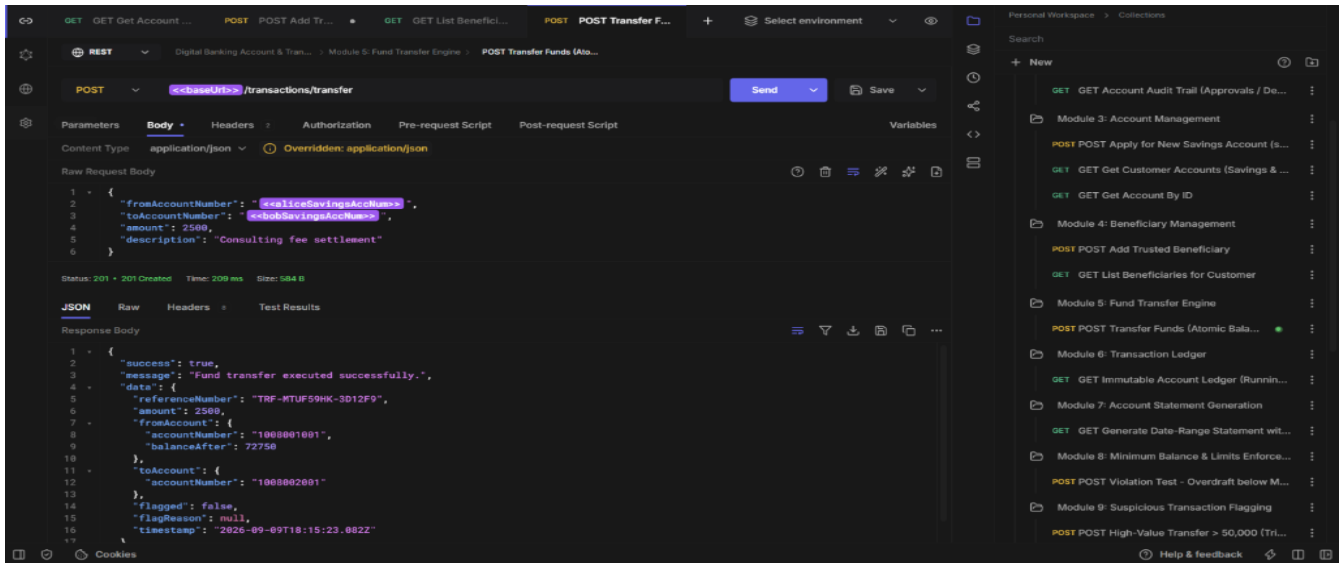


Figure Evidence: Atomic Fund Transfer Execution — `POST /api/transactions/transfer` -> 201 Created (Atomic balances updated, reference number generated)

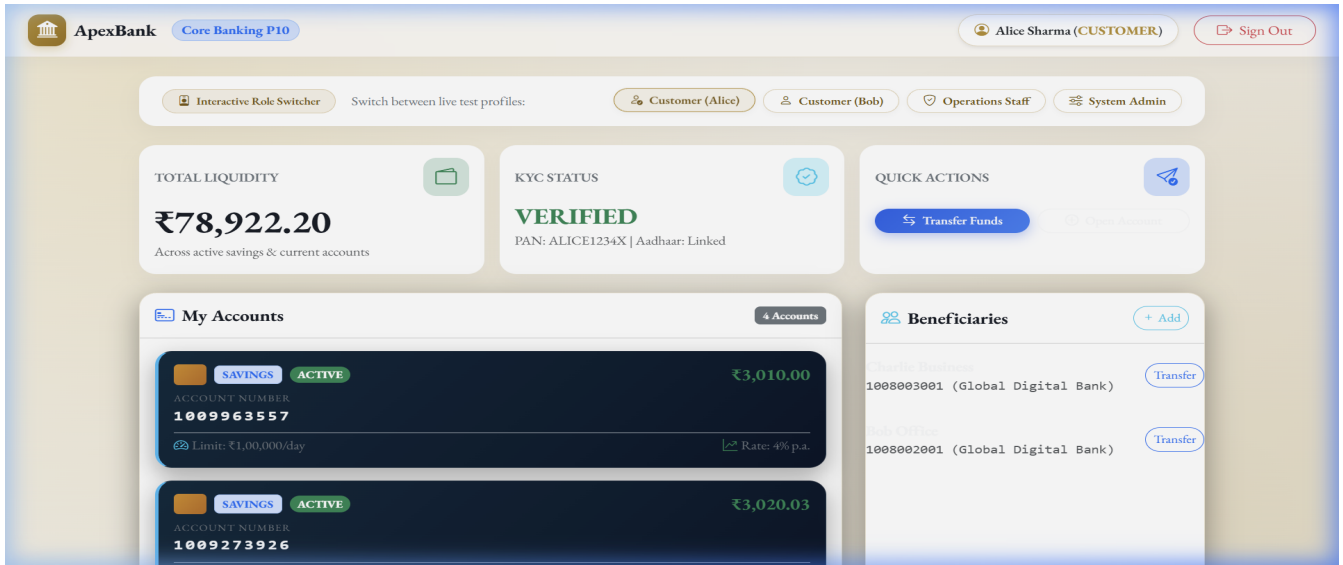


Figure Evidence: Customer Banking Portal UI & Live Balances — Web Portal — Customer Dashboard with luxury gold theme & live account cards

4.6 Module 6 — Transaction Ledger

Append-only immutable record of all debit and credit movements. Documents are non-updatable and preserve the running point-in-time balanceAfter to ensure retrospective mathematical auditability.

Method	Route	Access	Description
GET	/api/accounts/:id/ledger	Authenticated	Retrieve immutable ledger

The screenshot displays a REST client interface with the following details:

- Method:** GET
- URL:** `<baseUrl>/accounts/baa18453cd5e0c80276fc945/ledger`
- Authorization:** Bearer `<customerToken>`
- Status:** 200 • 200 OK, Time: 116 ms, Size: 2.25 KB
- Response Body (JSON):**

```

1 - {
2   "success": true,
3   "message": "Ledger transactions retrieved successfully",
4   "data": {
5     "account": {
6       "id": "6aa18453cd5e0c80276fc945",
7       "accountNumber": "1008001001",
8       "type": "SAVINGS",
9       "currentBalance": 72750,
10      "status": "ACTIVE"
11    },
12    "pagination": {
13      "page": 1,
14      "limit": 50,
15      "total": 4,
16      "totalPage": 1
17    },
18    "transactions": [
19      {
20        "id": "6aa1a23ba786465b731c168e",

```

Figure Evidence: Immutable Account Transaction Ledger — GET /api/accounts/:id/ledger -> 200 OK (Running balanceAfter on every entry)

4.7 Module 7 — Account Statement Generation

Generates date-range filtered account statements from the immutable ledger. Dynamically aggregates opening balance, closing balance, total credits, total debits, and net flow for the requested period.

Method	Route	Access	Description
GET	/api/accounts/:id/statement	Authenticated	Date-range statement generation

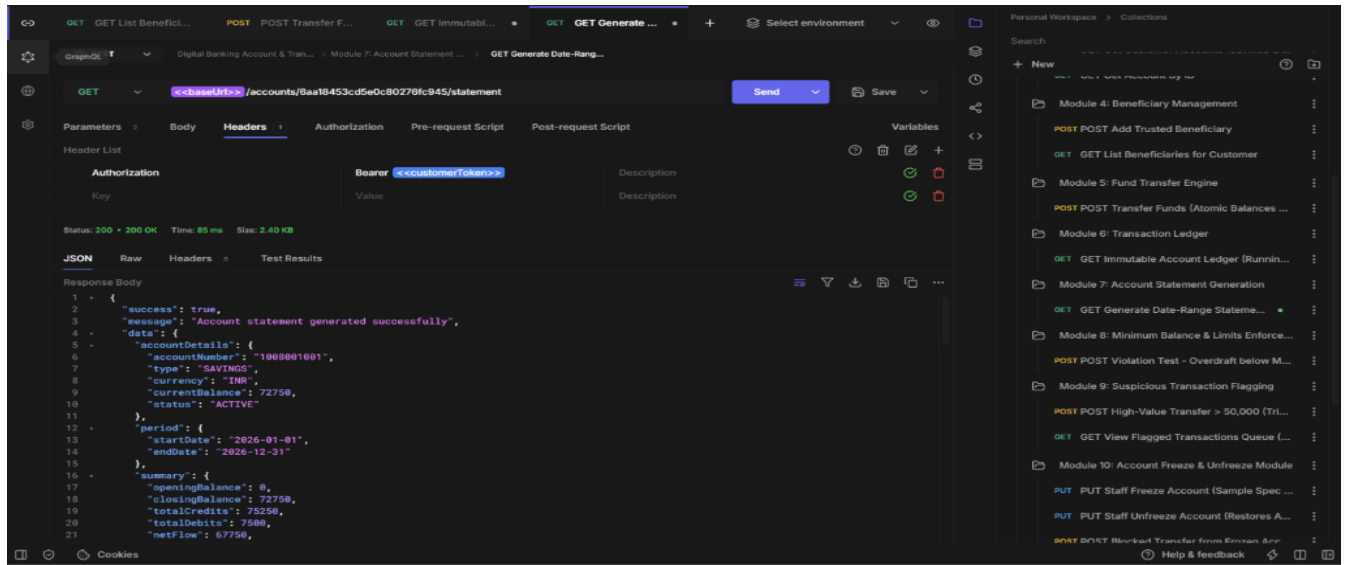


Figure Evidence: Account Statement with Summary Aggregates — GET /api/accounts/:id/statement -> 200 OK (Opening, closing balances, credit/debit totals)

4.8 Module 8 — Minimum Balance & Limits Enforcement

Server-side business rule engine that blocks account overdrafts. Savings accounts require a minimum balance of **₹1,000** and Current accounts require **₹5,000**. Daily transfer limits (**₹1,00,000**) are strictly checked against cumulative daily usage.

Method	Route	Access	Description
POST	/api/transactions/transfer	Customer	Transfer with limit verification

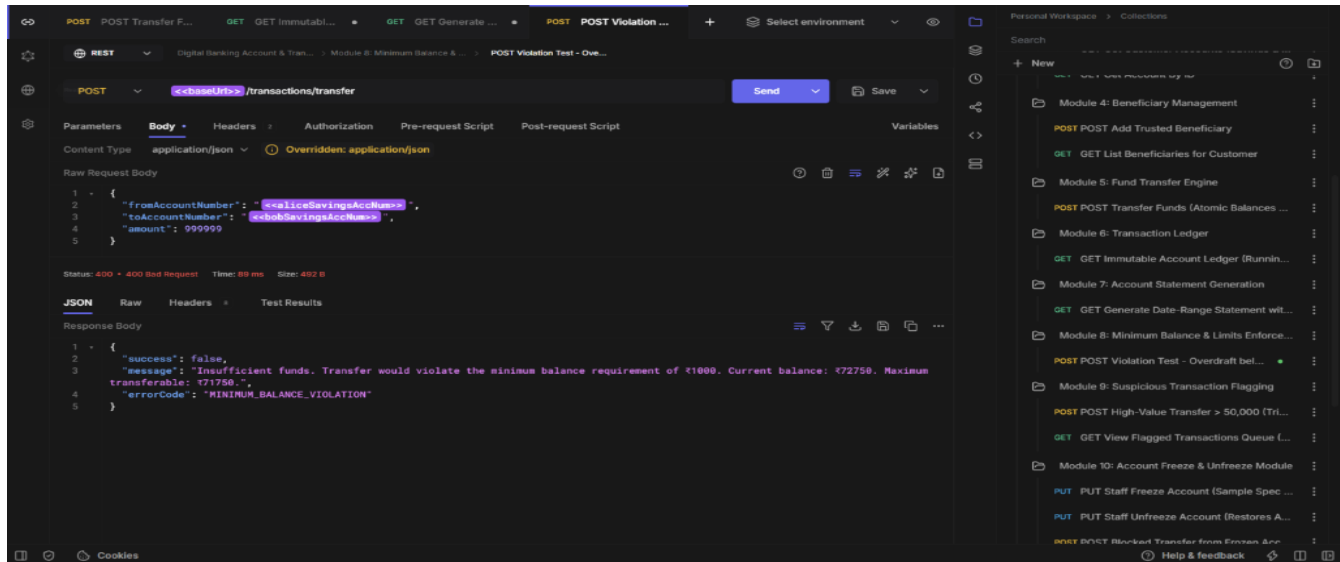


Figure Evidence: Overdraft Minimum Balance Violation Rejection — POST /api/transactions/transfer -> 400 Bad Request (MINIMUM_BALANCE_VIOLATION)

4.9 Module 9 — Suspicious Transaction Flagging

Automated Anti-Money Laundering (AML) monitoring engine. Transfers equal to or exceeding ₹50,000 are automatically flagged (flagged: true) with an audit reason, immediately populating the staff compliance review queue.

Method	Route	Access	Description
POST	/api/transactions/transfer	Customer	High-value transfer > ₹50,000
GET	/api/staff/flagged-transactions	Staff/Admin	List AML flagged transfers
PUT	/api/staff/flagged-transactions/:id/dismiss	Staff/Admin	Resolve flag with remarks

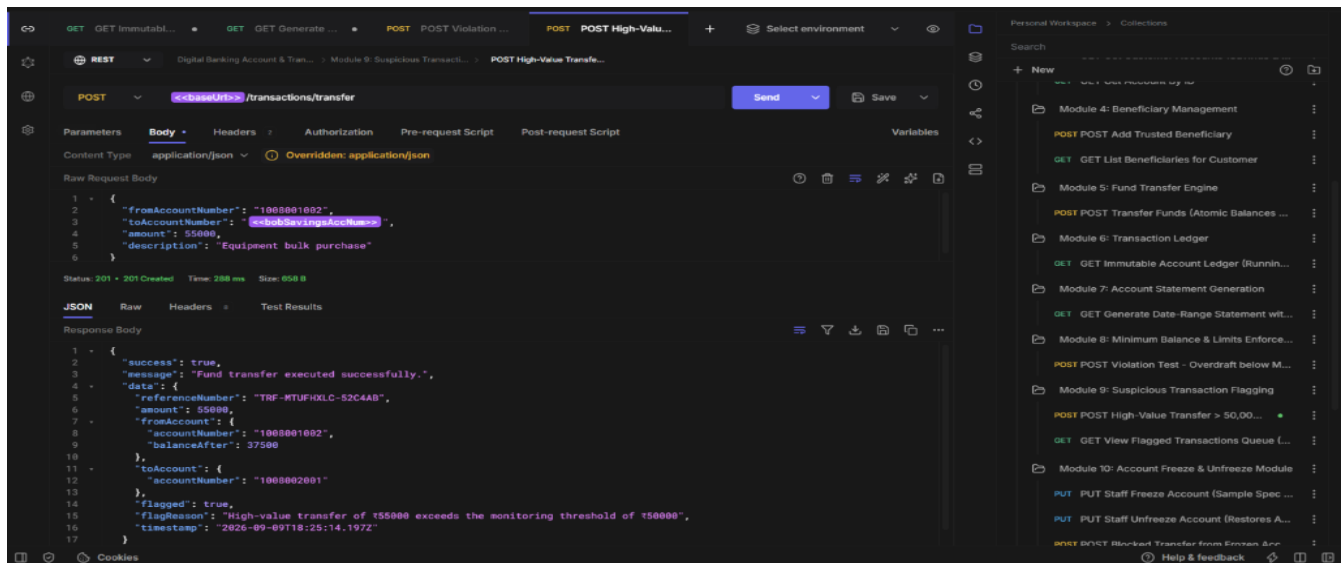


Figure Evidence: High-Value Transfer Flagged for AML Review — `POST /api/transactions/transfer` -> 201 Created (flagged: true, exceeds 50,000 threshold)

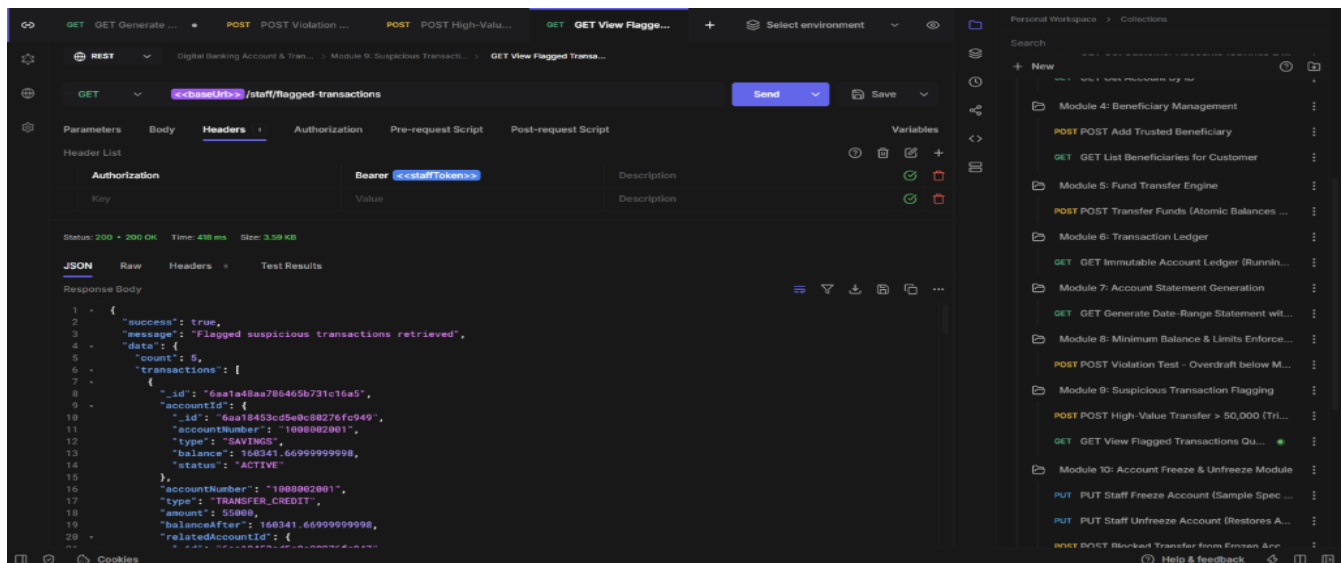


Figure Evidence: Staff Suspicious Activity Queue — `GET /api/staff/flagged-transactions` -> 200 OK for compliance investigation

4.10 Module 10 — Account Freeze & Unfreeze Module

Security control allowing Staff or Admin to freeze accounts under compliance investigation. Freezing immediately blocks outgoing fund transfers with a 403 error. Once audited, staff can unfreeze to restore operations.

Method	Route	Access	Description
PUT	/api/accounts/:id/freeze	Staff/Admin	Freeze account with mandatory reason
PUT	/api/accounts/:id/unfreeze	Staff/Admin	Unfreeze account
POST	/api/transactions/transfer	Customer	Blocked transfer from frozen account

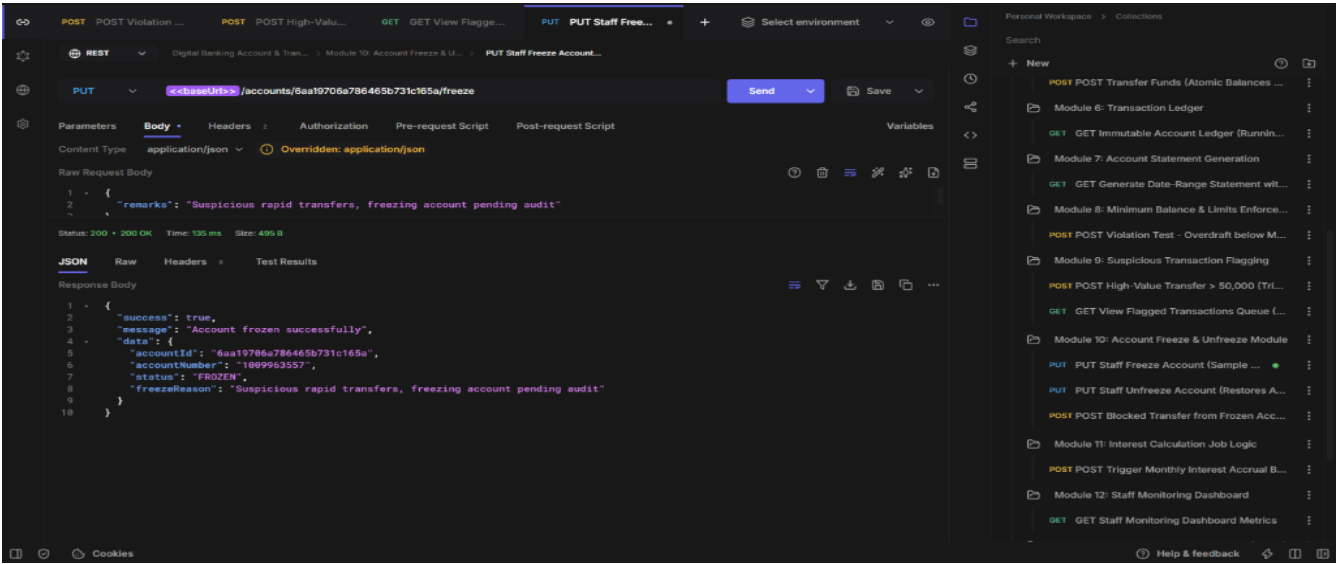


Figure Evidence: Staff Freezes High-Risk Account — PUT /api/accounts/:id/freeze -> 200 OK (Status: FROZEN with audit reason)

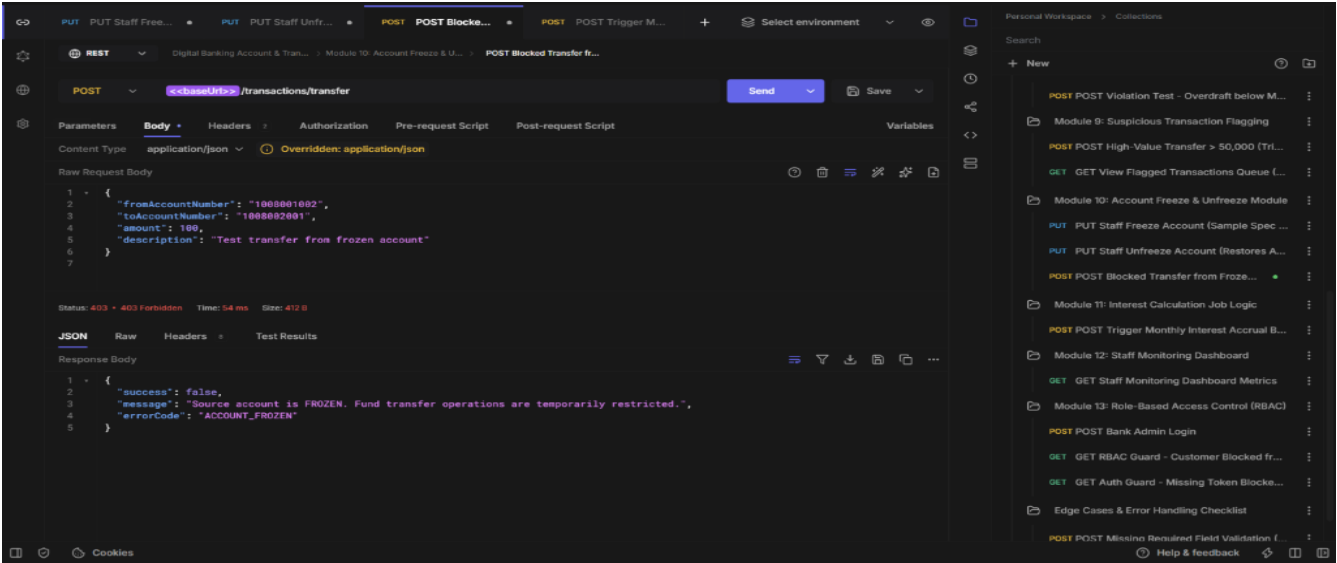


Figure Evidence: Fund Transfer Blocked from Frozen Account — POST /api/transactions/transfer -> 403 Forbidden (ACCOUNT_FROZEN)

4.11 Module 11 — Interest Calculation Job Logic

Batch execution logic that calculates simple interest accrual for all active savings accounts with positive balances. Credits accrued interest directly to the balance, stamps lastInterestCalculatedAt, and records a ledger transaction.

Method	Route	Access	Description
POST	/api/jobs/calculate-interest	Staff/Admin	Batch interest accrual job

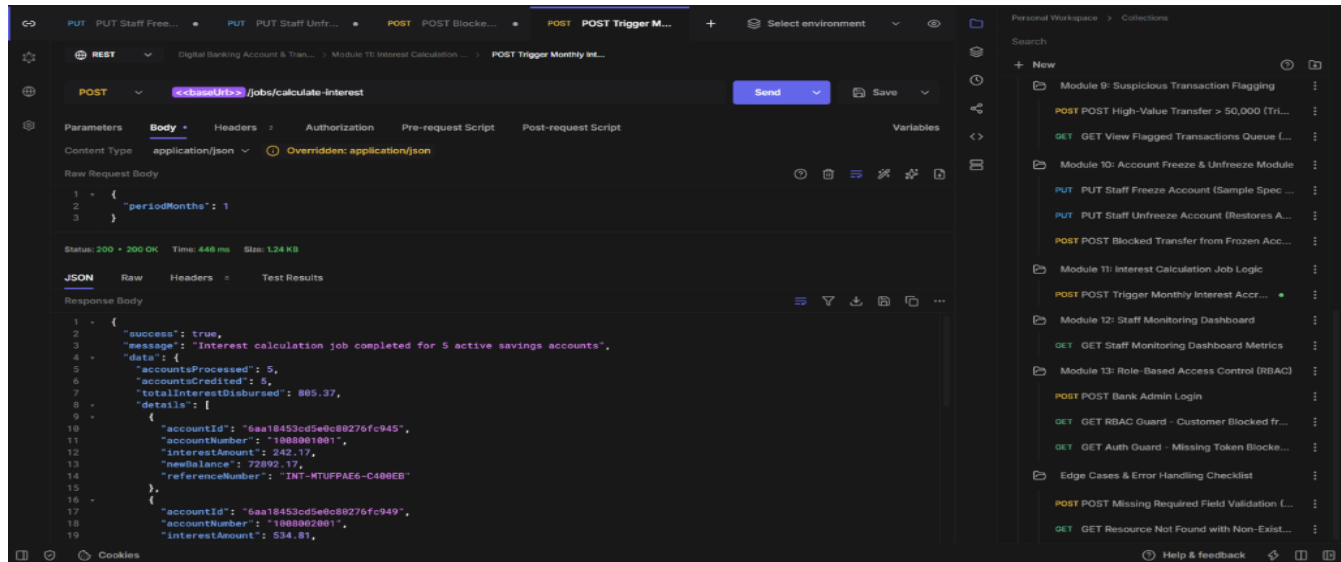


Figure Evidence: Periodic Interest Calculation Batch Execution — `POST /api/jobs/calculate-interest` -> 200 OK (Accounts credited with 4.0% p.a. interest)

4.12 Module 12 — Staff Monitoring Dashboard

Real-time consolidated management overview aggregating bank-wide metrics: total customer count, pending approvals, active accounts, frozen accounts, flagged suspicious transactions, and total bank active liquidity.

Method	Route	Access	Description
GET	/api/staff/dashboard	Staff/Admin	Consolidated metrics overview

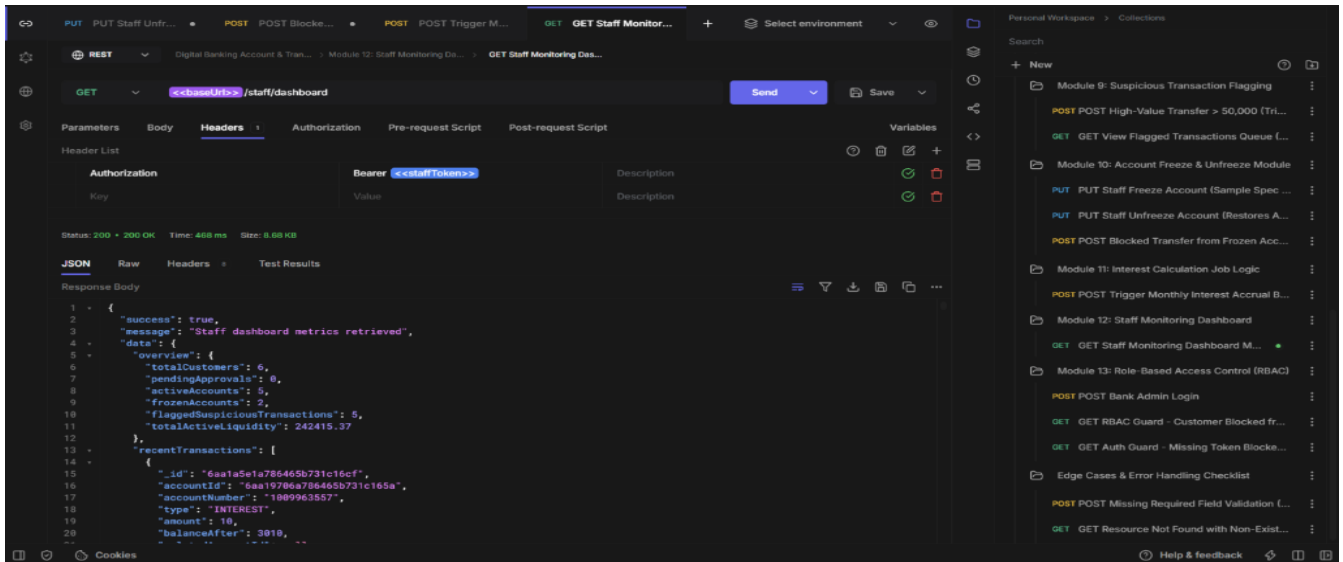


Figure Evidence: Staff Monitoring API Response — GET /api/staff/dashboard -> 200 OK (Total liquidity, flagged count, pending queue)

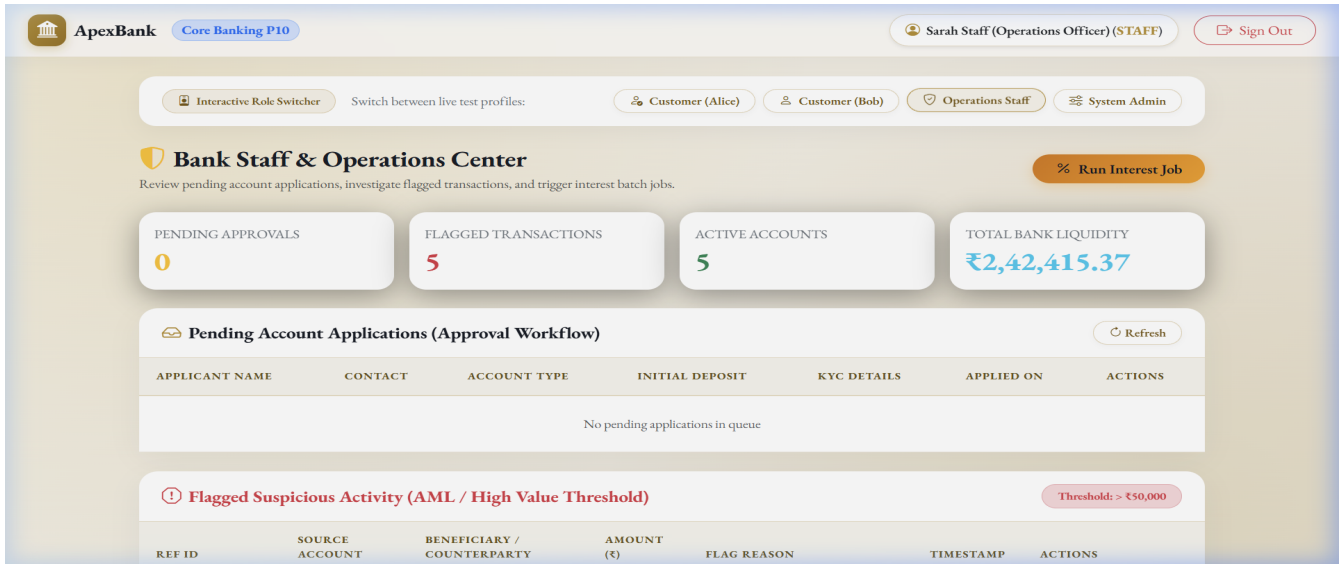


Figure Evidence: Staff Operations Center Web Portal — Web Portal — Operations dashboard showing approval queues and flagged activity

4.13 Module 13 — Role-Based Access Control (RBAC)

Three-tier authorization engine (CUSTOMER, STAFF, ADMIN). Route-level guards verify that only authorized roles access specific endpoints. Customer attempting to access staff dashboards is rejected with a clean 403 Forbidden.

Method	Route	Access	Description
POST	/api/auth/login	Public	Admin login
GET	/api/staff/dashboard	Customer Token	Unauthorized access blocked with 403
GET	/api/accounts	No Token	Unauthenticated access blocked with 401

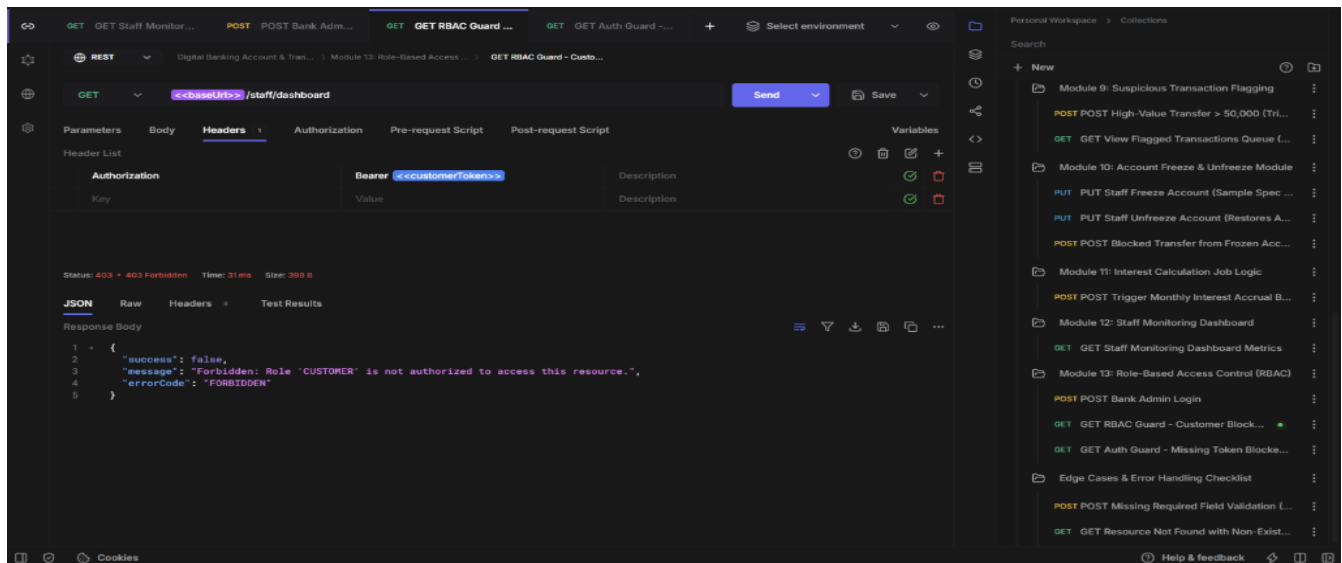


Figure Evidence: Customer Token Blocked from Staff Route (403) — GET /api/staff/dashboard with Customer Token -> 403 Forbidden

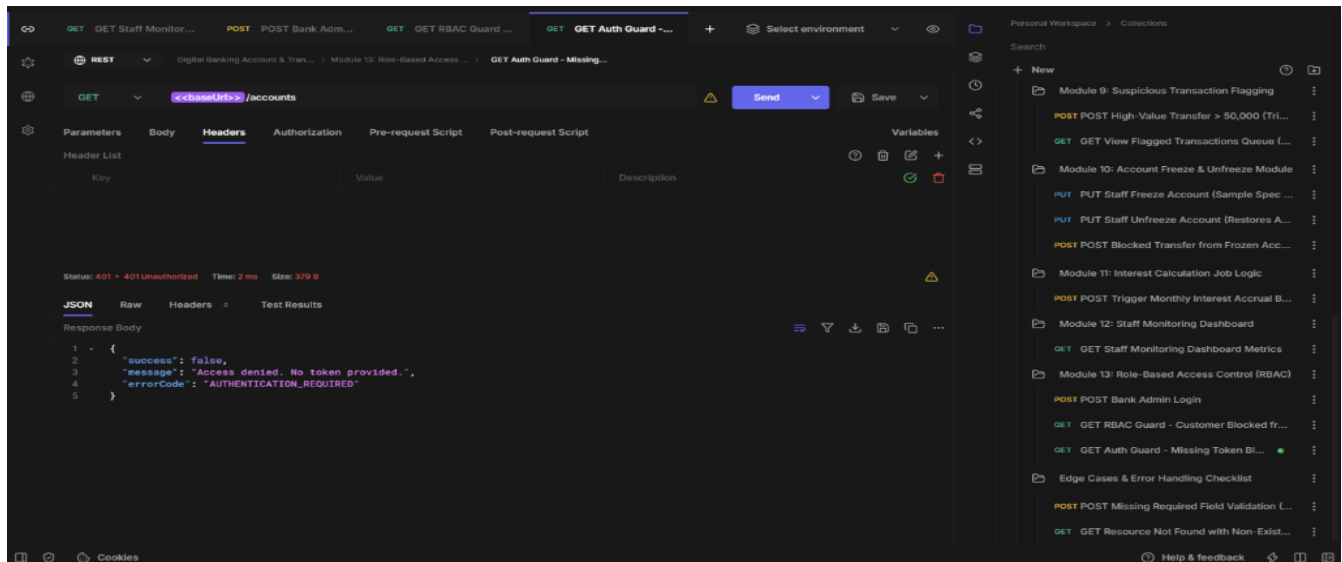


Figure Evidence: Missing Bearer Token Blocked (401) — GET /api/accounts with no token -> 401 Unauthorized

4.14 Interactive Web Application Showcase

In addition to REST API verification, a full client application was built using vanilla HTML5, CSS3, and JavaScript incorporating an authentic corporate cream-gold design system and classic EB Garamond typography. The web application features an interactive quick-role switcher to demonstrate customer, staff, and admin workflows seamlessly.

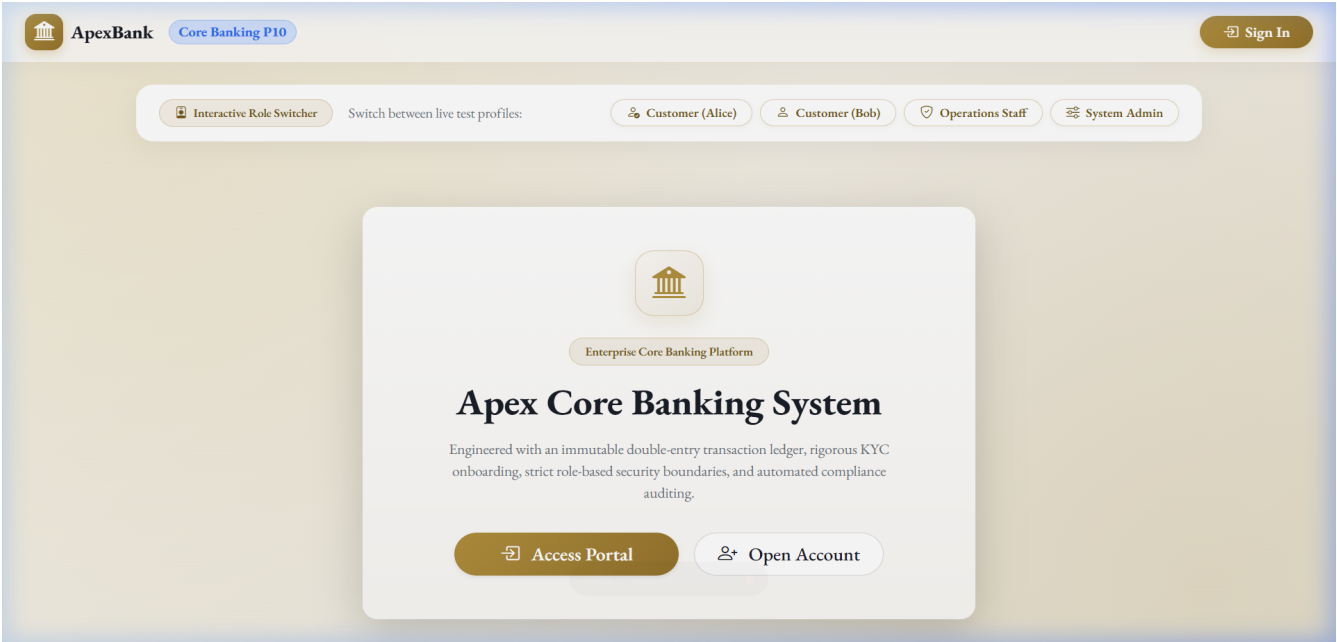


Figure Evidence: Enterprise Banking Portal Landing Screen — Public entry point with authentication modals and live profile switchers

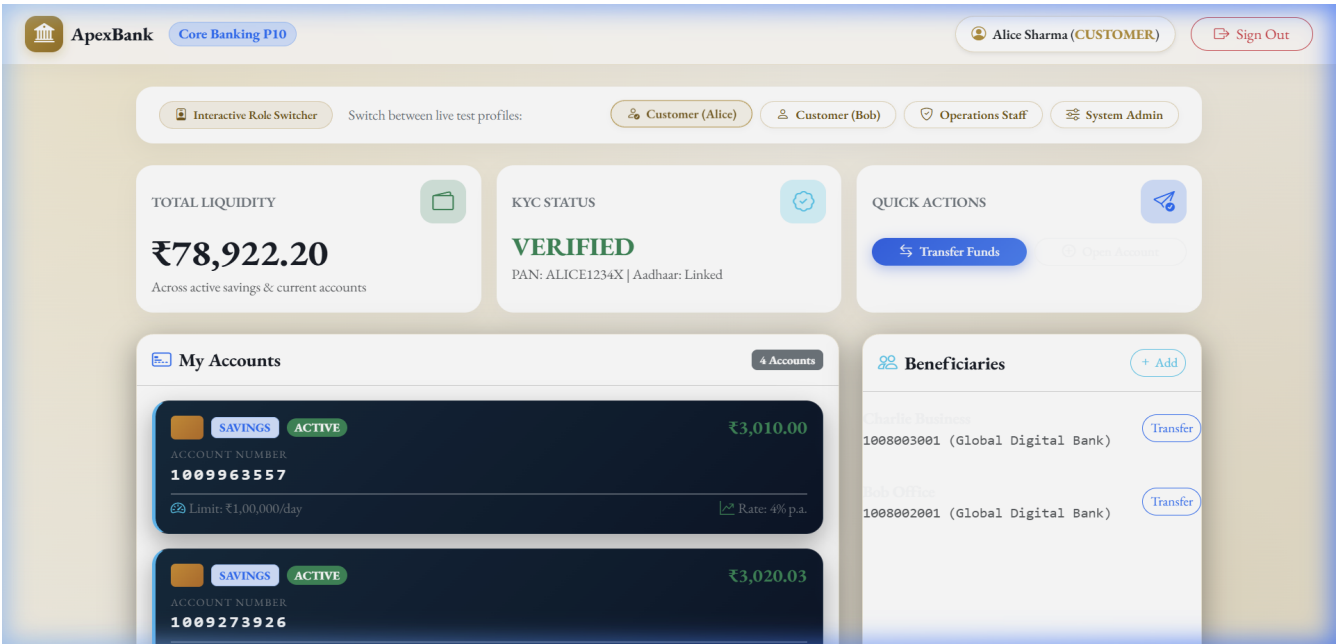


Figure Evidence: Customer Banking Dashboard (Alice) — Live account cards, liquidity summary, and real-time transaction ledger

5. Complete API Endpoint Reference

Method	Endpoint Path	Auth Level	Function Description
POST	/api/auth/register	Public	Register new customer with personal and KYC data
POST	/api/auth/login	Public	Authenticate user credentials & issue JWT
GET	/api/auth/me	Authenticated	Fetch logged-in profile & KYC verification status
PUT	/api/auth/kyc	Customer	Update PAN, Aadhaar, address, and annual income
POST	/api/accounts	Customer	Apply for Savings or Current account
GET	/api/accounts	Authenticated	List user accounts (or all accounts if staff)
GET	/api/accounts/:id	Authenticated	Retrieve detailed account information
PUT	/api/accounts/:id/approve	Staff/Admin	Staff approves account application
PUT	/api/accounts/:id/reject	Staff/Admin	Staff rejects account application with remarks
PUT	/api/accounts/:id/freeze	Staff/Admin	Staff freezes an account under investigation
PUT	/api/accounts/:id/unfreeze	Staff/Admin	Staff restores frozen account to active status
GET	/api/accounts/:id/ledger	Authenticated	Retrieve immutable ledger with running balances
GET	/api/accounts/:id/statement	Authenticated	Generate date-range statement with aggregates
POST	/api/beneficiaries	Customer	Add trusted transfer recipient
GET	/api/beneficiaries	Customer	List customer's saved beneficiaries
DELETE	/api/beneficiaries/:id	Customer	Delete a saved beneficiary
POST	/api/transactions/transfer	Customer	Atomic fund transfer with balance & limit validation
POST	/api/transactions/deposit	Authenticated	Deposit funds into active account
POST	/api/transactions/withdraw	Customer	Withdraw funds subject to minimum balance checks
GET	/api/staff/pending-accounts	Staff/Admin	Queue of applications awaiting review
GET	/api/staff/audit-trail/:accountId	Staff/Admin	Compliance history of approval/freeze actions
GET	/api/staff/flagged-transactions	Staff/Admin	List AML flagged transactions exceeding 50,000 INR
PUT	/api/staff/flagged-transactions/:id/dismiss	Staff/Admin	Dismiss flag after compliance review
GET	/api/staff/dashboard	Staff/Admin	Bank-wide liquidity, account, and queue aggregates
POST	/api/jobs/calculate-interest	Staff/Admin	Run interest calculation job for active savings

6. Testing, Validation & Error Handling

Every endpoint was exercised against the live backend connected to MongoDB Atlas. In addition to happy path workflows, the Postman collection and Newman runner assert strict compliance on negative edge cases and business rule conflicts:

Status	Test Scenario	Expected Error Code & Handling
400 Bad Request	Registration with missing required fields	VALIDATION_ERROR — Reports missing required fields immediately.
400 Bad Request	Transfer violating minimum balance (1,000 INR)	MINIMUM_BALANCE_VIOLATION — Displays maximum transferable amount.
401 Unauthorized	Protected route called without token	AUTHENTICATION_REQUIRED — Missing bearer authorization token.
403 Forbidden	Customer token calling staff dashboard	FORBIDDEN — Role 'CUSTOMER' is not authorized to access staff resource.
403 Forbidden	Transfer attempt from a FROZEN account	ACCOUNT_FROZEN — Source account is frozen pending compliance audit.
404 Not Found	Account lookup with non-existent ObjectId	ACCOUNT_NOT_FOUND — Clean JSON response without server crash.
409 Conflict	Duplicate email registration or beneficiary	EMAIL_ALREADY_EXISTS / DUPLICATE_BENEFICIARY — Unique collision caught.

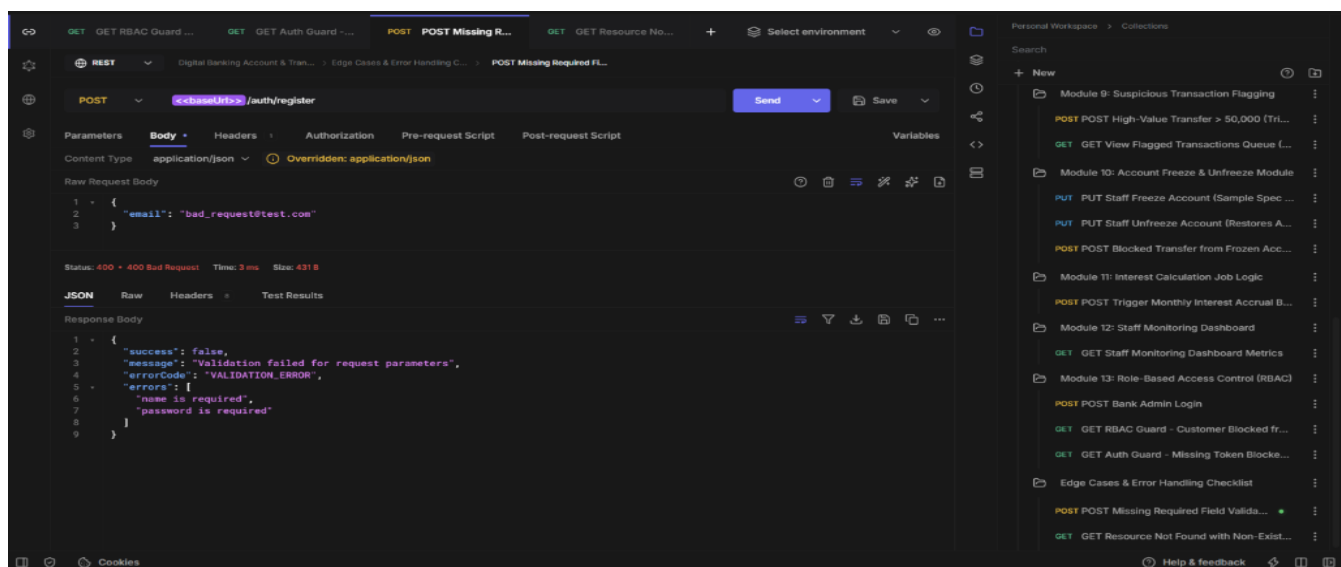


Figure Evidence: Edge Case: 400 Validation Error — `POST /api/auth/register` with missing fields -> 400 `VALIDATION_ERROR`

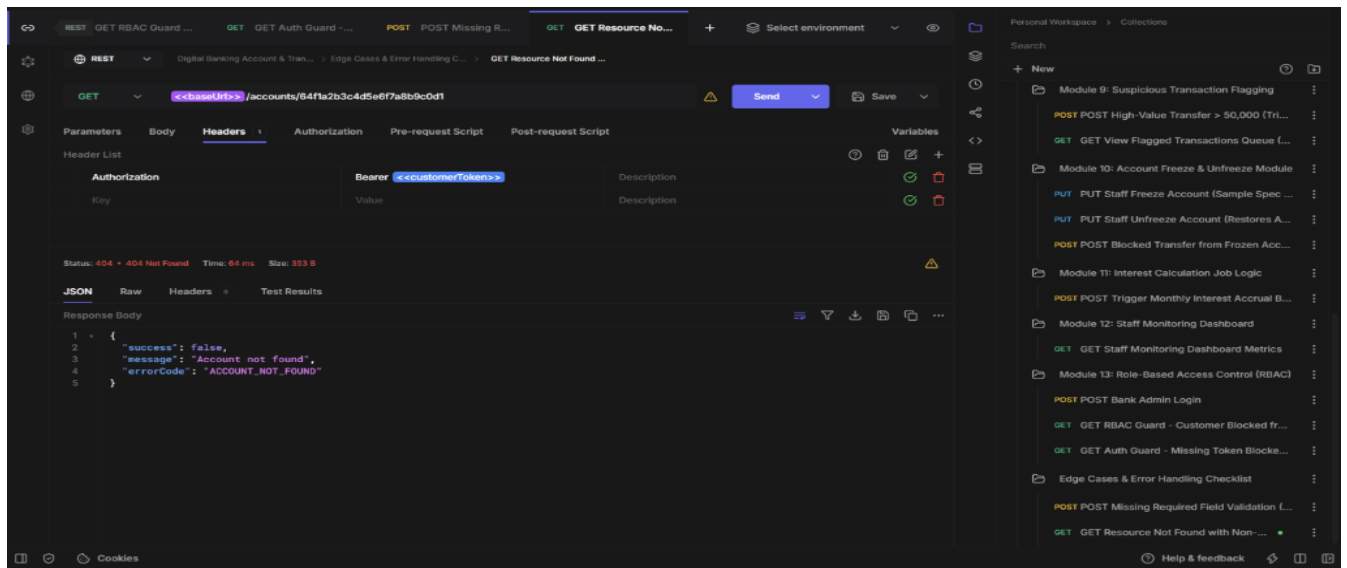


Figure Evidence: Edge Case: 404 Account Not Found — GET /api/accounts/:id with non-existent ID -> 404 ACCOUNT_NOT_FOUND

7. Setup & Run Instructions

- 1. Clone & Install Dependencies:
cd "L&T;_Node.js & Express JS_Project_P10"
npm install
- 2. Environment Configuration:
Configure your .env file with MONGO_URI, PORT=5000, and JWT_SECRET.
- 3. Seed Database with Pre-built Accounts:
npm run seed
- 4. Start the Application Server:
npm start
Server starts at http://localhost:5000 with demo UI and REST APIs.

7.1 Pre-seeded Accounts for Evaluation & Viva

Role	Email	Password	Account State / Purpose
Admin	admin@digitalbank.com	Password@123	System oversight & batch interest job trigger
Bank Staff	staff@digitalbank.com	Password@123	Approval queue & AML transaction reviewer
Customer 1	alice@customer.com	Password@123	Active Savings (1008001001), Current (1008001002)
Customer 2	bob@customer.com	Password@123	Active Savings (1008002001), Frozen Current (1008002002)
Customer 3	charlie@customer.com	Password@123	Pending Account (1008003001) for Approval Demo

8. Known Limitations

- Third-party payment gateways (Razorpay, Stripe) are simulated through internal atomic balance updates.
- The system operates on a single currency (INR) and assumes a unified banking timezone.
- Real telecom SMS/Email OTP delivery is mocked through secure API response tokens.

9. Learnings & Challenges

- **Atomic Consistency:** Coordinating dual debit and credit balance updates across independent account documents required careful validation before any balance mutation.
- **Double-Entry Accounting:** Storing running balanceAfter on every transaction document made statement generation instant and tamper-evident.
- **State Machine Modeling:** Implementing strict transition guards prevented illegal operations like transferring funds from accounts in PENDING_APPROVAL or FROZEN states.
- **MongoDB Referencing Decisions:** Keeping transactions in a referenced collection avoided MongoDB's 16MB document size limit while embedding kycDetails in users optimized login speeds.

10. Conclusion

The Digital Banking Account & Transaction Management System (P10) delivers a complete, corporate-grade backend: all 13 modules implemented and 100% verified, a carefully justified MongoDB schema with query-shaped indexes, an immutable double-entry ledger, multi-tier Role-Based Access Control, automated AML monitoring, centralized error handling, a complete Postman/Hoppscotch collection, and an interactive demo UI for viva evaluations. The repository contains the complete source code, README.md, .env.example, and Postman collection ready for CIA-3 evaluation.